

Pandas

Pandas是基于NumPy 的一种工具，该工具是为解决数据分析任务而创建的。Pandas 纳入了大量库和一些标准的数据模型，提供了高效地操作大型数据集所需的工具。Pandas提供了大量能使我们快速便捷地处理数据的函数和方法。你很快就会发现，它是使Python成为强大而高效的数据分析环境的重要因素之一。

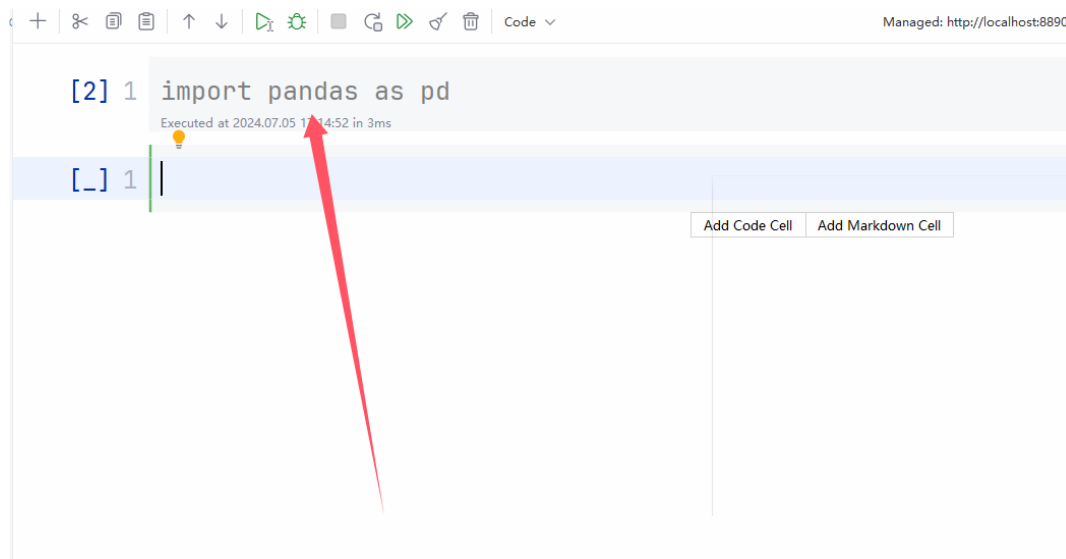
Pandas中一共有三种数据结构，分别为：Series、DataFrame和MultiIndex（老版本中叫Panel）。其中Series是一维数据结构，DataFrame是二维的表格型数据结构，MultiIndex是三维的数据结构。

官方网址：<https://pandas.pydata.org/>

需要下载后才能使用，下载命令如下：

```
1 pip install pandas
2 pip install -i https://pypi.tuna.tsinghua.edu.cn/simple/ pandas
```

下载完成后，导入包，进行运行，无报错，则证明下载成功：



1.Series数据结构

Series是一个类似于一维数组的数据结构，它能够保存任何类型的数据，比如整数、字符串、浮点数等。主要由一组数据和与之相关的索引两部分构成。

Series	
index	value
0	12
1	4
2	8
3	16

基本操作

```
1 #pd.Series(data=None,index=None,dtype=None)
2 #data=传入的数据，可以ndarray、list等
3 #index=索引，必须是唯一的，且与数据的长度相等，如果不传入索引，就会默认从0-N创建整数索引
4 #dtype=数据的类型
```

```
1 import numpy as np
2 import pandas as pd
3
4 #1.指定内容,默认索引
5 pd.Series(np.arange(10))
6 #2.指定内容，指定索引
7 pd.Series(np.arange(5), index=[1, 2, 3, 4, 5])
8 #3.通过字典数据创建
9 color = pd.Series({'red': 100, 'blue': 200, 'green': 500, 'yellow': 1000})
10 print(color)
11 #Series属性-#获取索引
12 print(color.index )
13 #Series属性-#获取数值
14 print(color.values)
15 #通过索引获取数值
16 print(color.iloc[2])
```

2.DataFrame基础操作

DataFrame是一个类似于二维数组或表格(如excel)的对象，既有行索引，又有列索引。

行索引，表明不同行，横向索引，叫index，0轴，axis=0。

列索引，表明不同列，纵向索引，叫columns，1轴，axis=1。

DataFrame			
index	writer	title	price
0	mark	cookbook	23.56
1	barkat	HTML5	50.70
2	tom	Python	12.30
3	job	Numpy	28.00

```

1 pd.DataFrame(data=None,index=None,columns= None)
2 #data=传入的数据，可以ndarray、list等
3 #index=行标签，如果没有传入索引参数，则默认自动创建一个从0-N的整数索引。
4 #columns=列标签，如果没有传入索引参数，则默认自动创建一个从0-N的整数索引。

```

1.基础操作

列如：生成10名同学，5门课程的数据。

```

1 #生成10名同学，5门课程的数据
2 score=np.random.randint(40,100,(10,5))
3 print(score)
4 print(type(score))#<class 'numpy.ndarray'>
5 score_df=pd.DataFrame(score)
6 print(score_df)
7 print(type(score_df))#<class 'pandas.core.frame.DataFrame'>

```

2.增加行、增加列

```

1 #构造行索引序列
2 subjects = ["语文", "数学", "英语", "政治", "体育"]
3 #构造列索引序列
4 students=['同学'+ str(i) for i in range(score_df.shape[0])]
5 print(students)
6 #构造数据
7 data=pd.DataFrame(score,columns=subjects,index=students)
8 data

```

	语文	数学	英语	政治	体育
同学0	78	85	94	66	89
同学1	62	64	61	45	62
同学2	95	95	85	88	44
同学3	55	83	72	77	47
同学4	71	68	86	45	96
同学5	90	46	65	66	77
同学6	49	58	80	62	72
同学7	48	75	86	53	54
同学8	80	40	73	99	81
同学9	54	95	56	72	44

3.DataFrame的属性

```
1 print(data.shape)#(10, 5)
2 print(data.index)
3 print(data.columns)
4 print(data.values)
5 print(data.T)#转置
6 print(data.head())#默认显示前5行内容,
7 print(data.tail())#默认显示后5行内容
```

4.DataFrame索引的设置

修改行列索引值

```
1 #修改行列索引值
2 students=['学生_'+str(i) for i in range(score_df.shape[0])]
3 subjects=['Python','Java','C++','GO','R']
4 #必须整体全部修改
5 data.index=students
6 data.columns=subjects
7 print(data)
```

重设索引

```
1 #reset_index(drop=False) 设置新的下标索引
2 #drop=False: 默认为False, 不删除原来的索引, 如果为True, 则删除原来的索引
3 print(data.reset_index())
4 print(data.reset_index(drop=True))
```

set_index(keys,drop=True)以某列值设置为新的索引。

keys: 表示列索引名或者列索引名的列表

drop: 默认为True。删除原来的列

```
1 #以月份设置为新的索引
2 df.set_index('month')
```

```
1 #设置多个索引, 以年和月份
2 df.set_index(['year','month'])
```

3.DataFrame数据操作

1.读取数据

```
1 #读取文件
2 data=pd.read_csv("stock_day.csv")
3 #可以删除一些不需要的列数据，方便操作
4 data=data.drop(["ma5","ma10","ma20","v_ma5","v_ma10","v_ma20"],axis=1)
5 data
```

2.索引操作

获取某一天的收盘价（close）的结果

```
1 #1.直接使用行列索引（先列后行）
2 print(data['close']['2018-02-14'])#收盘价格
3 print(data['open']['2018-02-14'])#开盘价格
```

```
1 #2.使用iloc或者loc使用索引的方式
2 #使用loc只能指定行列索引的名字
3 data.loc['2018-02-14':'2018-02-22','close']
4 #使用iloc可以通过索引的下标获取数据
5 data.iloc[:3,:5]#获取前3天，前5列的结果
```

```
1 #3.使用loc进行下标和名称组合做索引
2 data.loc[data.index[0:4],['open','close','high','low']]
```

3.排序

```
1 #按照开盘价大小进行排序
2 data.sort_values(by='open',ascending=False).head()#ascending=False表示降序
3 data.sort_values(by='open',ascending=True).head()#ascending=True表示升序
4 #按照多个标签进行排序
5 data.sort_values(by=['open','high'],ascending=False)#ascending=False表示降序
```

sort_index给索引进行排序

```
1 #原始数据的股票索引是从大到小的，现重新排序，从小到大
2 data.sort_index()
```

4.DataFrame运算

1.算术运算

1	运算符	操作函数	说明
2	+	add()	加法
3	-	sub()	减法
4	*	mul()	乘法
5	/	div()	除法
6	//	floordiv()	取整
7	**	pow()	乘方
8	%	mod()	取余

```
1 data['open'].add(10)
2 data['open'].sub(10)
3 data['open'].mul(2)
4 data['open'].div(2)
5 data['open'].floordiv(2)
6 data['open'].pow(2)
7 data['open'].mod(2)
```

2.逻辑运算符号

```
1 data['open']>23
2 data[data['open']>23].head()
```

5.DataFrame统计运算

1.describe 叙述统计，综合分析

```
1 data.describe()
```

	open	high	close	low	volume	price_change	p_change	turnover
count	643.000000	643.000000	643.000000	643.000000	643.000000	643.000000	643.000000	643.000000
mean	21.272706	21.900513	21.336267	20.771835	99905.519114	0.018802	0.190280	2.936190
std	3.930973	4.077578	3.942806	3.791968	73879.119354	0.898476	4.079698	2.079375
min	12.250000	12.670000	12.360000	12.200000	1158.120000	-3.520000	-10.030000	0.040000
25%	19.000000	19.500000	19.045000	18.525000	48533.210000	-0.390000	-1.850000	1.360000
50%	21.440000	21.970000	21.450000	20.980000	83175.930000	0.050000	0.260000	2.500000
75%	23.400000	24.065000	23.415000	22.850000	127580.055000	0.455000	2.305000	3.915000
max	34.990000	36.350000	35.210000	34.010000	501915.410000	3.030000	10.030000	12.560000

2.统计函数

```
1 #0代表求列的结果。1表示求行的结果
2 #最小最大值
3 data.min(0)
4 data.max(0)
5 #方差
6 data.var(0)
7 #标准差
8 data.std(0)
9 #中位数
10 data.median(0)
11 #众数
12 data.mode(0)
13 #求出最大值的位置
14 data.idxmax(axis=0)
15 #求出最小值的位置
16 data.idxmin(axis=0)
```

3.累计统计函数

```
1 data1=data['close']
2 data1.cumsum()#累计、累加
3 data1.cummax()#计算前n个最大值
4 data1.cummin()#计算前n个最小值
5 data1.cumprod()#计算前n个积
```

6.DataFrame文件读取与存储

1.read_csv

```
1 data2=pd.read_csv("stock_day.csv")
2 data2.head()
```

2.to_csv

```
1 #保存“open”列前10行数据为新的一个csv文件
2 data3=data2["open"].head(10)
3 data3.to_csv("day0706.csv",index=False)
```

